Scrapy 2.19 Cheat Sheet

Python 3.11+ · `pip install scrapy` · docs.scrapy.org

PROJECT & CLI

```
scrapy startproject myproj && cd myproj
scrapy genspider [-t basic|crawl|csvfeed|xmlfeed] \
  name example.com
scrapy crawl name -O out.json   # -o appends
scrapy crawl name -o out.csv:csv
scrapy crawl name -a cat=books -s LOG_LEVEL=INFO
scrapy runspider spider.py -O out.jsonl
scrapy shell "https://example.com"
scrapy fetch | view | parse <url>
scrapy parse --spider=NAME -c cb <url>
scrapy list | check NAME | bench
scrapy settings --get NAME | version -v
scrapy genrequest 'curl ...'
scrapy crawl big -s JOBDIR=crawls/1
```

JOBDIR: Ctrl-C once to pause, rerun to resume. Global: `-L LEVEL` `--nolog` `--pdb`

SPIDER

```
import scrapy
class Q(scrapy.Spider):
    name = "q"
    allowed_domains = ["example.com"]
    start_urls = ["https://example.com/"]
    custom_settings = {"DOWNLOAD_DELAY": 1}

    async def start(self):  # optional
        yield scrapy.Request(self.start_urls[0])

    def parse(self, response):  # or async def
        for q in response.css("div.quote"):
            yield {"text": q.css("span::text").get()}
        yield from response.follow_all(
            css="a.next", callback=self.parse)
```

Generic: `CrawlSpider` + `Rule(LinkExtractor(allow=r"/p/"), callback="parse_item", follow=True)`, `SitemapSpider`, `XMLFeedSpider`, `CSVFeedSpider`.

SELECTORS

`response.css("h1::text").get()`	first / None
`.getall()`	all matches
`css("a::attr(href)")`	attribute
`xpath("//h1/text()")`	XPath
`.re(r"\d+")`, `.re_first()`	regex
`response.json()`	JSON body
`response.urljoin(href)`	absolute URL

REQUESTS

`Request(url, callback, errback, method, headers, cookies, meta, cb_kwargs, dont_filter, priority)`

`FormRequest.from_response(resp, formdata={})` · `JsonRequest(url, data={})` · `response.follow(link, cb)`

meta: `dont_redirect` `dont_retry` `handle_httpstatus_list` `proxy` `download_timeout` `max_retry_times` `cookiejar`

ITEMS & PIPELINES

```
class P(scrapy.Item):
    name = scrapy.Field()

class Clean:
    def process_item(self, item):
        return item   # raise DropItem to discard

ITEM_PIPELINES = {"myproj.pipelines.Clean": 300}
```

Lower number runs first. Items: dict, dataclass, attrs, `scrapy.Item` (via `ItemAdapter`). Media: `FilesPipeline`, `ImagesPipeline`.

KEY SETTINGS (DEFAULT)

Setting	Default
CONCURRENT_REQUESTS	16
CONCURRENT_REQUESTS_PER_DOMAIN	8*
DOWNLOAD_DELAY	0*
ROBOTSTXT_OBEY	False*
DOWNLOAD_TIMEOUT	180 s
RETRY_TIMES	2
AUTOTHROTTLE_ENABLED	False
HTTPCACHE_ENABLED	False
DEPTH_LIMIT / CLOSESPIDER_ITEMCOUNT	0 (off)
LOG_LEVEL	DEBUG
TWISTED_REACTOR_ENABLED	True

*New projects set ROBOTSTXT_OBEY=True, PER_DOMAIN=1, DELAY=1.

Precedence: `-s` › spider `custom_settings` › `settings.py` › add-ons › defaults. Disable built-in component: set to `None`.

```
FEEDS = {"out/%(name)s.jsonl":
  {"format": "jsonlines", "overwrite": True}}
```

FROM A SCRIPT

```
from scrapy.crawler import AsyncCrawlerProcess
from scrapy.utils.project import get_project_settings
p = AsyncCrawlerProcess(get_project_settings())
p.crawl("q")
p.start()  # blocks
```

DATA FLOW

Spider `start()` → spider mw → **Engine** → **Scheduler** (dupefilter) → downloader mw → **Downloader**/handlers → response → downloader mw → spider mw → callback → items → **Pipelines** · requests → Scheduler

CONTRIBUTING (REPO)

```
pip install -e . tox pre-commit
pre-commit install
tox -e py -- tests/test_crawl.py
pre-commit run --all-files
tox -e mypy; tox -e pylint; tox -e docs
```

Tests: mock server, `example.com` only, parametrize, `build_from_crawler()`. Don't edit `docs/news.rst`. Never rebase a reviewed PR.